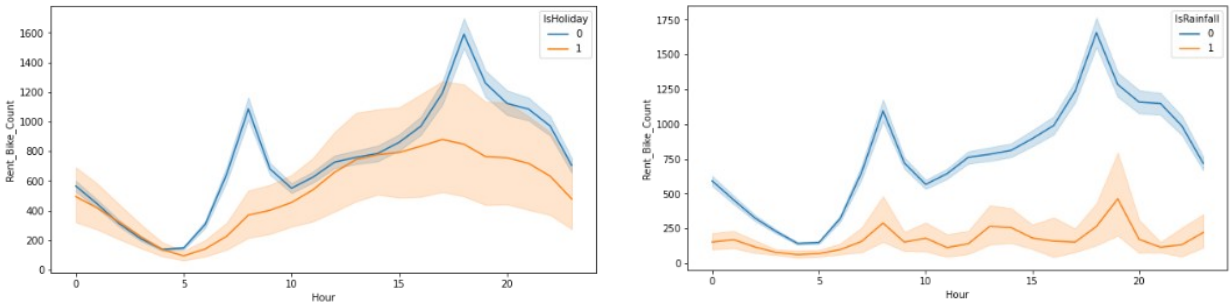
Project Report
IE 465
Md Zahidul Islam and Alex Hines

Background: This report will discuss the approach our group took to solve a bike demand regression task using data from Seoul, South Korea. The goal is to use several different input values from the data set to train different machine learning algorithms capable of making accurate, reliable predictions for the number of bikes rented every hour. This problem is important because it would allow a company to know how many customers they may be expecting throughout the day. Accurate demand prediction can help the company determine the number of staff needed, the number of bikes that need prepared for renting, etc.

Objectives: To predict the number of bikes rented every hour based on several different weather conditions, date conditions, holiday conditions, etc. To find out the best machine learning model suitable for the dataset.

Data: The dataset [1] comes from a company based in Seoul, South Korea. Bike renting in Seoul is extremely popular compared to many other parts of the world, and these companies can become very overloaded with customers. The data set contains one year's worth of data collected hourly. This results in about 8,620 observations. At each data point, we have the Date, Hour, Temperature, Humidity, Wind Speed, Visibility, Dew Point Temperature, Solar Radiation, Rainfall, Snowfall, Seasons, Functioning Day, and Bikes Rented. The first 12 values are used as inputs for the model to predict the final variable "Bikes Rented".

Methods: Our group performed several different pre-processing techniques to prepare the data for input into the model. We noticed that we had a complete data set without any missing values. We then calculated a correlation coefficient matrix of correlations between our inputs and outputs. We then converted quite a few continuous values (decimal numbers) to categorical values. In this case, we changed visibility, rainfall, snowfall, solar radiation, and holiday to "IsVisible" "IsRainfall" "IsSnowfall" "IsSolarRadiation" and "IsHoliday". All these values previously had continuous values but were instead changed to 0 or 1. In this case, a '1' represents it is visible outside (visibility value is low) is rainfall (is raining), etc. This process allowed our model to differentiate between the weather outside quicker and more effectively than simply using continuous values. We also split the data into training and testing sets using the SKLearn toolbox in Python. 80% of the values were used for training and 20% were used for testing. Fig. 1 depicts the relation between the response (rent bike count) and all the other predictors. It is evident that all the predictors influence the response and hour is a really important predictor and other predictors also have a good impact on the response. This is shown in the Fig.1 below.



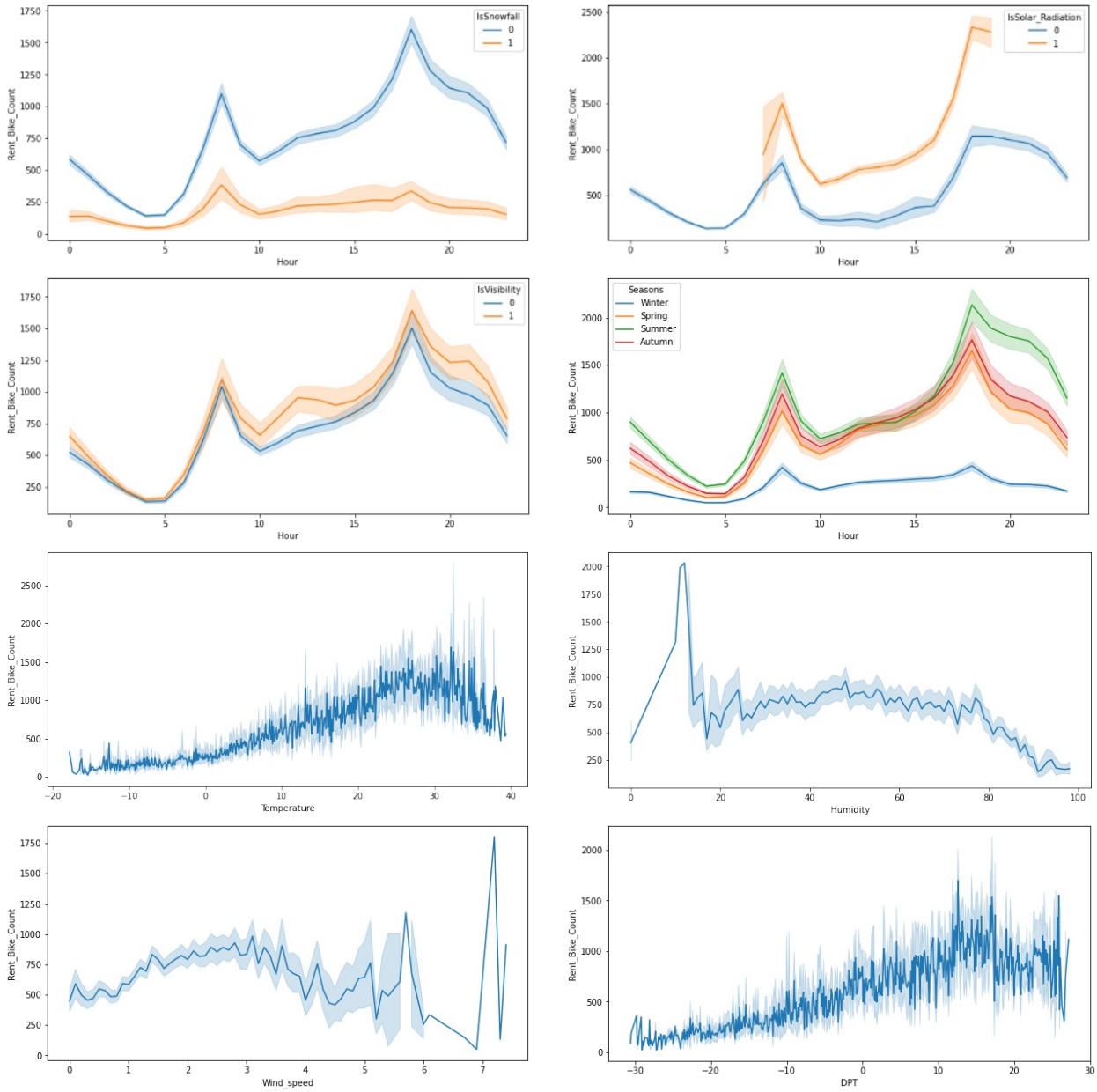


Figure 1: Influence of different variables on the response (Rent Bike Count)

Our group chose to use different machine learning models to compare and determine which performs the best. In this research, we looked at using Linear Regression (LR), Artificial Neural Network (ANN), Gaussian Process Regression (GPR), Long Short-Term Memory (LSTM), XGBoost, and Random Forest (RF). We decided to use linear regression to provide a simple baseline for model comparison. The other models are well known in the machine learning field and are great for regression tasks, some are specific for time-series data (GPR and LSTM). Originally, we used default hyperparameters to train the initial models. We recorded results of a number of different error metrics (R^2, Root Mean Squared Error (*RMSE*), Mean Absolute Error (*MAE*)) and then determined that we would perform hyper-parameter tuning on the top three models. All results are reflected in the model comparison table in the results section of this report.

The structure of each layer in ANN was as follows: Fully connected linear layer, ReLU activation function, batch normalization, and dropout. We trained the ANN model with an increasing number of hidden layers and found that the R^2 value between the predicted values and the actual values increases both

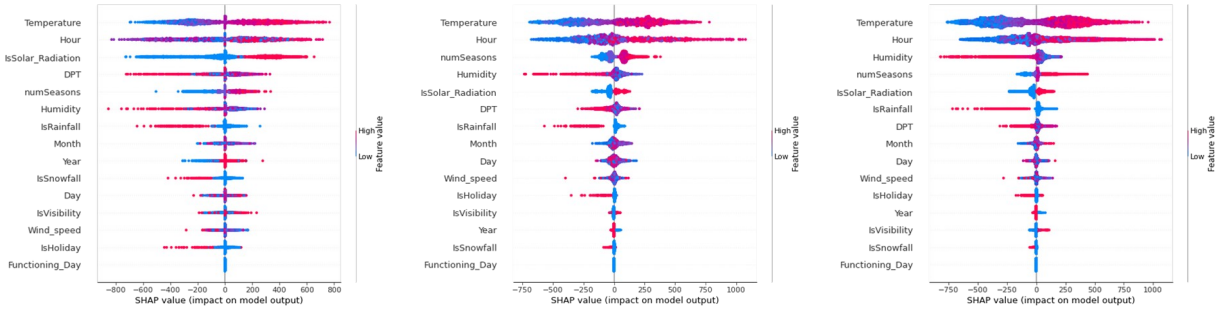
in the training set and testing set with an increase in the number of hidden layers. We used the "Ray Tune" PyTorch package for hyperparameter tuning and let the number of neurons in each layer, dropout probability, and learning rate vary. The optimization criteria is Kling-Gupta efficiency (KGE). After evaluating 128 random samples from the search space, the best network configuration is [100, 1000, 100, 100, 1000], learning rate: 0.03869, and dropout probability: 0.00509. The final R^2 values for the training set and the testing set are 99% and 96% respectively. We used GridSearchCV for the hyperparameter tuning of RF model. We used three-fold cross validation, and tuned n_estimators {300, 400, …, 900}, max_features {2, 3, …, 10}, max_depth {10, 20, …, 100}, min_samples_split {1, 2, 3, 4}, min_samples_leaf {2, 3, 4}, and bootstrap {True, False}. The best parameters are as follows- n_estimators: 500, min_samples_split: 2, min_samples_leaf: 2, max_features: 8, max_depth: 90, bootstrap: False. In case of the XGBoost model, we used Sequential Grid Search and used 10-fold cross-validation with the optimization criteria being the RMSE value. First, max-depths {5-14}, then, subsamples {0.1, 0.2, …, 1}, colsample_bytree {0.1, 0.2, …, 0.5}, colsample_bylevel {0.1, 0.2, …, 0.5} are optimized. Finally, we optimize learning rate {. 0.001, 0.00316228, 0.01, 0.03162278, 0.1}. The tuned hyperparamets for XGBoost are - colsample_bylevel: 0.5, colsample_bytree: 0.5, importance_type: gain, learning_rate: 0.01, max_depth: 9, n_estimators: 3438, subsample: 0.9. For training the LSTM model, we kept the last two month's data as the testing set. The sequence size is 730 (last one month's data) and the batch size is 96. We kept only one hidden layer with 100 neurons for the LSTM model. The dropout is 0.2 and the learning rate is 0.05.

Results and Discussion: As previously mentioned, we decided to use a number of different error metrics to compare our models. We chose to compare R^2 values, *RMSE* values, and *MAE* values for model comparison. Table 1 compares all models that were trained and tested (using the same values for each model).

Algorithm	R_Squared	RMSE	MAE
Linear Regression	0.72	338.56	258.53
GPR	0.85	246.94	224.42
ANN	**0.96**	**142.17**	**86.7**
Random Forest	0.88	238.64	161.36
XGBoost	0.89	212.1	139.17
LSTM	**0.95**	**151.24**	**87.29**

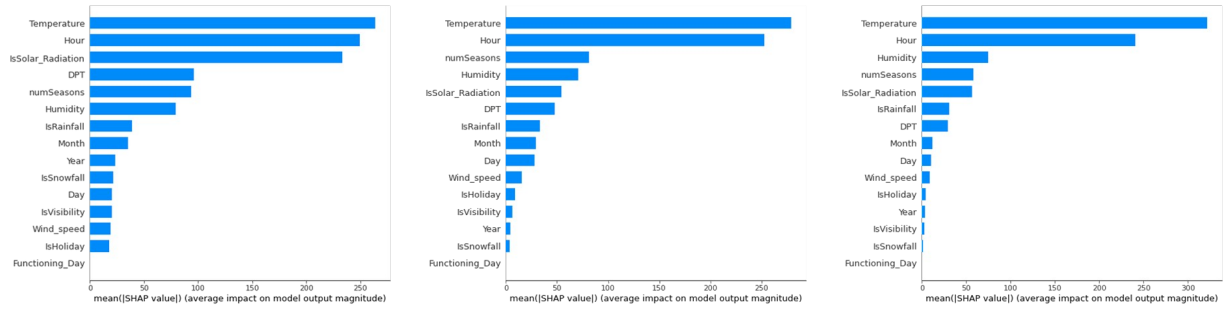
Table 1: Model Comparison Table

As we can see, the ANN model resulted in the highest R^2 value among all the models. The model also had the lowest *RMSE* and *MAE* values. This means that the ANN is our optimal model in this case. The LSTM model performed the second best for every metric. As a result, we consider this model to be the second best of the models that were tested. These two models were by far the best two performers. The R^2 value ANN model is also greater than the study of Sathishkumar [2,3]. Their highest R^2 value of 95%. Therefore, our models are working good compared to the other works on the same dataset.



ANN XGBoost RF

Figure 2: Shapley Values for different models



ANN XGBoost RF

Figure 3: Feature importance for different models

Figures 2 and 3 show the Shapley values and feature importance for ANN, XGBoost, and RF models, respectively. From the Shapley values of temperature, we can see that the higher temperature value is also increasing the bike rent count. It makes sense and the other shapely values are also giving consistent. From the feature/variable importance plot, it is evident that Hour, Temperature, NumSeasons, Humidity, and IsSoalrRadiation are the most important features because they are the most five important features for all three models.

Conclusion: In conclusion, our group believes that we were able to successfully predict bikes rented for the South Korean dataset that we were researching. Our value for the ANN model is lower than any previous research that we have seen in papers discussing this dataset. We would recommend the company utilize the tuned ANN if they were interested in predicting bike demand for their company. Future researchers can test the ANN model on the datasets extracted from other cities to see if a similar conclusion can be drawn.

References

[1] UCI Machine Learning Repository. (2020). Seoul Bike Sharing Demand Data Set. [Data file]. Retrieved from https://archive.ics.uci.edu/ml/datasets/Seoul+Bike+Sharing+Demand

[2] Sathishkumar V E, Jangwoo Park, and Yongyun Cho. 'Using data mining techniques for bike sharing demand prediction in metropolitan city.' *Computer Communications*, Vol.153, pp. 353-366, March 2020.

[3] Sathishkumar V E and Yongyun Cho. 'A rule-based model for Seoul Bike sharing demand prediction using weather data.' *European Journal of Remote Sensing*, pp. 1-18, February 2020.